

Automated Feature Engineering for Contextual Bandits via AATention Transfer Learning

(Technical Appendix)

Ethan Pedersen
Computer Science Department
Brigham Young University
Provo, United States
ethanp55@byu.edu

Jacob W. Crandall
Computer Science Department
Brigham Young University
Provo, United States
crandall@cs.byu.edu

I. CODE

We provide our code in the supplementary “code” folder. There is a README.md file that describes the code layout. Our language of choice was Python 3.10. The AATention network was implemented with TensorFlow 2.16.2 [1]. We used the default multi-head attention implementation from TensorFlow; whenever we applied multi-head attention, we used four heads. For our dense (i.e., fully connected) layers and attention key dimensions, we used a dimension size of 128, with the exception of the final dense layer (output dimension of 100). For processing text, we used a maximum length of 100 tokens and an embedding size of 64. For dropout layers, we used a rate of 0.1. K-Nearest Neighbors was implemented with scikit-learn 1.4 [2].

II. TEXT LAYER ARCHITECTURE

The text layer is a custom layer and is one of the first in the AATention network. It receives as input a string of text that represents the generator description (the generator the network should make predictions for) and a string of text that represents the domain/environment description (the domain the generator is operating in). The text layer tokenizes and embeds the strings, adds positional encodings to both, performs self-attention on both, and outputs two self-attention matrices. The architecture of this custom layer is shown in Figure 1.

III. STATE LAYER ARCHITECTURE

The AATention network also receives as input a numeric vector s_t representing the current state of the environment at time t . The vector is masked to account for varying dimension sizes (see the previous section), passed through a dense layer, and finally passed through a custom state attention layer. This custom layer applies positional encodings and self-attention to the masked and transformed state vector and outputs a self-attention vector. The architecture of this custom layer can be found in Figure 2.

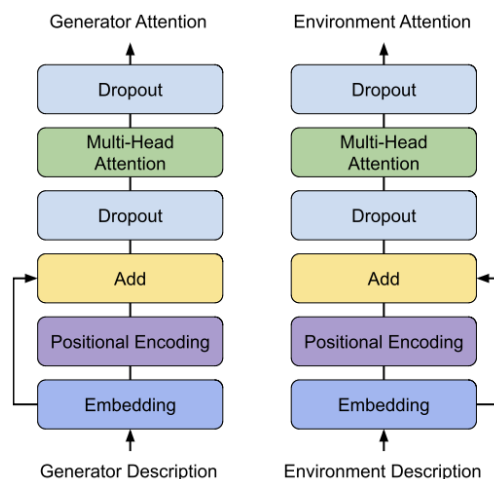


Fig. 1: Text layer architecture.

IV. CROSS ATTENTION LAYER ARCHITECTURE

A custom cross attention layer operates on the processed strings and state vectors. This layer performs cross attention between (1) the generator description self-attention matrix (one of the outputs from the custom text layer) and the state self-attention vector and (2) the environment description self-attention matrix (the second output from the custom text layer) and the state self-attention vector. It might seem somewhat strange to perform attention between a matrix and a vector, and the efficacy is questionable, but the motivation behind this process was to allow the network to learn how generator and environment descriptions relate to the current state. The vectors from the two cross-attention executions are added with the original state self-attention vector that was passed as input; the resulting vector is returned. The specific architecture of this layer can be found in Figure 3.

V. REPEATED GAMES PAYOFF MATRICES

The payoff/reward matrices we used for the repeated, two-player prisoner’s dilemma, chicken game, and coordination

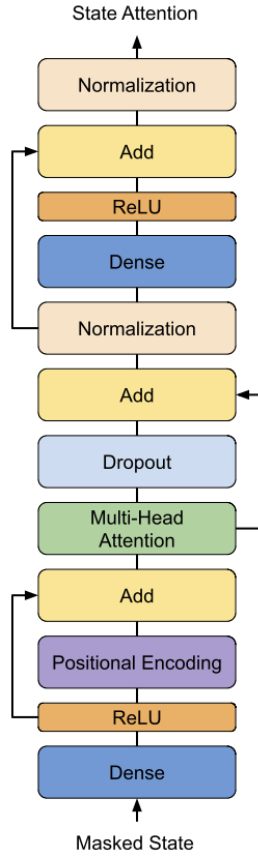


Fig. 2: State attention layer architecture.

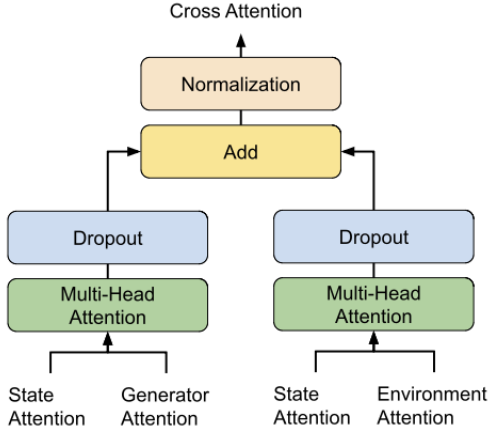


Fig. 3: Cross attention layer architecture.

game can be found in Tables I, II, and III, respectively. Note that the actions for the coordination game are A and B as they do not have any natural interpretation, other than the fact that players must “coordinate” to pick the same action.

VI. ADAPTABILITY SCORE

The adaptability score is formed using two different measures of regret: *competitive regret* (R_{comp}) and *cooperative regret* (R_{coop}). Competitive regret measures how well an

		Player 2	
		Cooperate	Defect
Player 1	Cooperate	(3, 3)	(-3, 5)
	Defect	(5, -3)	(-1, -1)

TABLE I: Repeated prisoner’s dilemma payoff matrix.

		Player 2	
		Swerve	Straight
Player 1	Swerve	(0, 0)	(-1, 3)
	Straight	(3, -1)	(-5, -5)

TABLE II: Repeated chicken game payoff matrix.

algorithm performs in scenarios when other agents are not inclined to cooperate. Formally,

$$R_{\text{comp}} = U_d^* - \bar{U}_d, \quad (1)$$

where U_d^* is the highest expected reward the agent can receive when paired with non-cooperative associates and $\bar{U}_d$ is the average reward the agent actually receives when interacting with these same agents. A competitive score (denoted $\mathcal{S}_{\text{comp}}$) is derived from normalized competitive regret as follows:

$$\mathcal{S}_{\text{comp}} = 1 - \frac{R_{\text{comp}}}{U_d^* - U_{\min}}, \quad (2)$$

where $U_{\min}$ is the lowest expected reward an agent could receive. A competitive score near one indicates that the algorithm performs effectively when placed in environments with non-cooperative associates, while a competitive score near zero indicates less effective performance in this regard.

Cooperative regret, which measures how well an algorithm performs when paired with other algorithms that are inclined to cooperate, is measured similarly. Formally, cooperative regret is given by:

$$R_{\text{coop}} = U_c^* - \bar{U}_c, \quad (3)$$

where U_c^* is the expected payoff when all agents cooperate and $\bar{U}_c$ is the agent’s average reward when interacting with cooperative associates. A cooperative score (denoted $\mathcal{S}_{\text{coop}}$) is derived from normalized cooperative regret as follows:

$$\mathcal{S}_{\text{coop}} = 1 - \frac{R_{\text{coop}}}{U_c^* - v_{\text{mm}}}, \quad (4)$$

where v_{mm} is the agent’s maximin value in the scenario.

Ideally, an agent should have both high competitive and cooperative scores. To measure this, we compute the *adaptability score*:

$$\mathcal{S}_A = \min(\mathcal{S}_{\text{comp}}, \mathcal{S}_{\text{coop}}). \quad (5)$$

Our justification for basing the adaptability score on the minimum (rather than, for example, the average) is to distinguish algorithms that adapt effectively to different situations from those that do well only in a single scenario. Low performance in at least one area indicates a learning deficiency.

		Player 2	
		A	B
Player 1	A	(2, 2)	(0, 0)
	B	(0, 0)	(2, 2)

TABLE III: Repeated coordination game payoff matrix.

VII. ADDITIONAL FIGURES

For the sake of space in the main paper, we only showed certain figures. In this section, we present every figure. Same as the figure found in the main paper, Figure 4 shows the four algorithms’ average performances, with standard errors, in the five domains when AATention was trained on JHG data. Similarly, Figure 5 shows average performances and standard errors when AATention was trained on chicken game data. Figure 6 shows average performances and standard errors when AATention was trained on both JHG and chicken game data. Finally, Figures 7-10 show the adaptability scores for each algorithm in the prisoner’s dilemma, chicken game, coordination game, and Grid Stag Hunt domain, respectively (we did not calculate scores for the forex domain due to its more complex nature) when the AATention network was trained on data from both the JHG and repeated chicken game; each of these figures show the algorithms’ average defect, cooperate, and adaptability scores, respectively.

VIII. DELIMITATIONS

Though we tried to be thorough in the work presented in this paper, we recognize that there are certain areas we did not fully explore, which remain items of future work. One might be able to obtain better results under different AATention network architectures, which is not something we experimented with. Related to this point, we used a relatively small text embedding dimension within the network due to limited computational resources. We also only experimented with training on two domains and testing in five domains; this leads to certain unanswered questions, such as: How might AATention performance change if it were trained on more domains?

REFERENCES

- [1] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin, S. Ghemawat, I. Goodfellow, A. Harp, G. Irving, M. Isard, Y. Jia, R. Jozefowicz, L. Kaiser, M. Kudlur, J. Levenberg, D. Mané, R. Monga, S. Moore, D. Murray, C. Olah, M. Schuster, J. Shlens, B. Steiner, I. Sutskever, K. Talwar, P. Tucker, V. Vanhoucke, V. Vasudevan, F. Viégas, O. Vinyals, P. Warden, M. Wattenberg, M. Wicke, Y. Yu, and X. Zheng, “TensorFlow: Large-scale machine learning on heterogeneous systems,” 2015, software available from tensorflow.org. [Online]. Available: <https://www.tensorflow.org/>
- [2] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

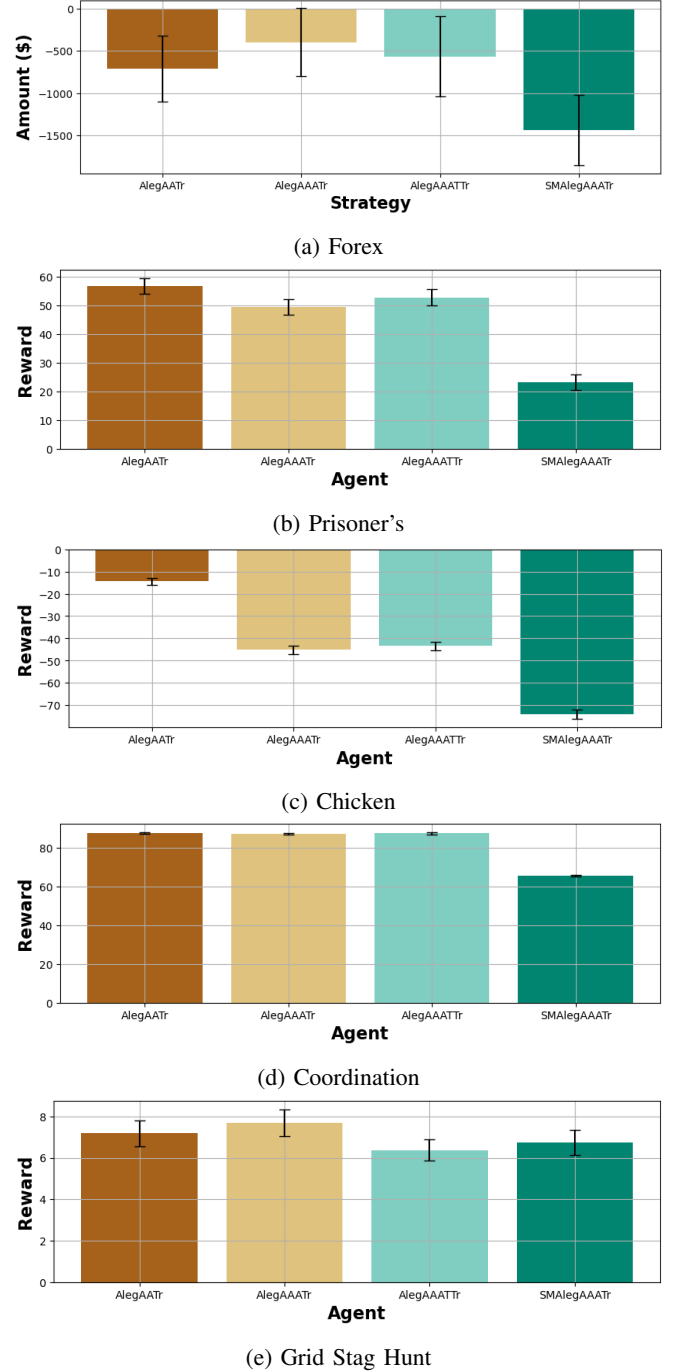
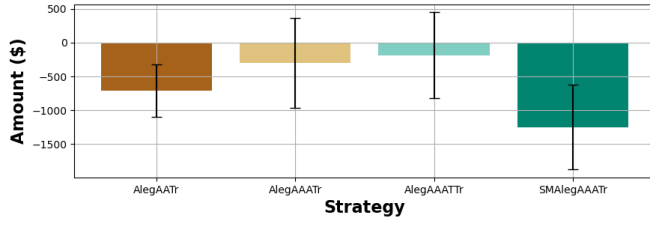
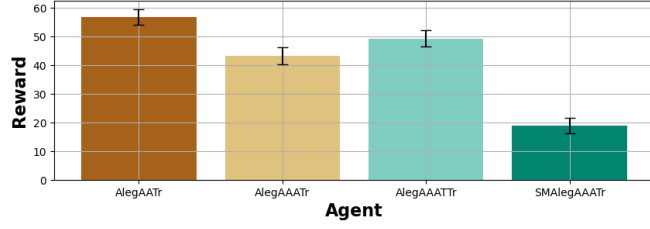


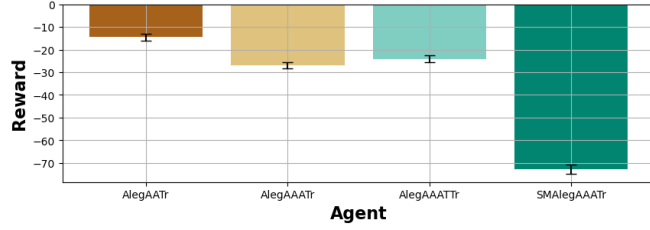
Fig. 4: Average results for each algorithm across the evaluation domains when AATention was trained only on JHG data. Bars indicate standard errors.



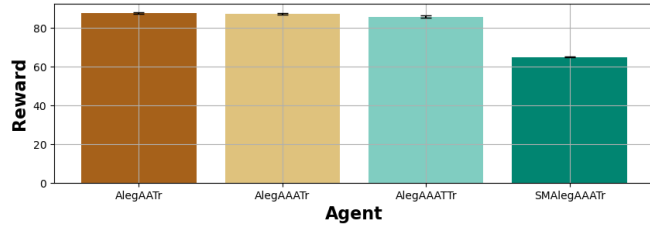
(a) Forex



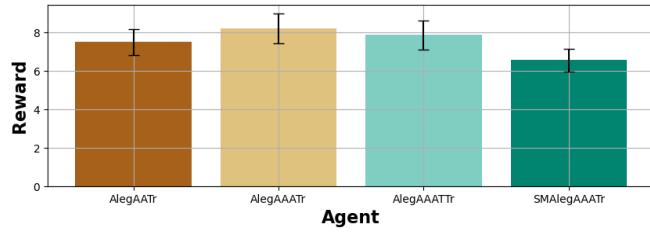
(b) Prisoner's



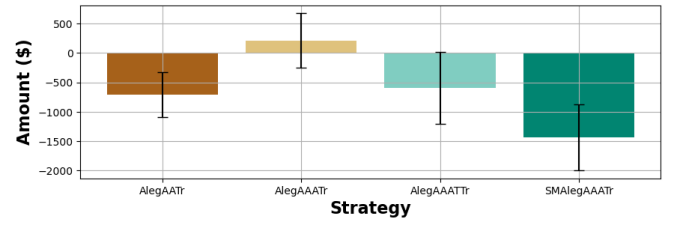
(c) Chicken



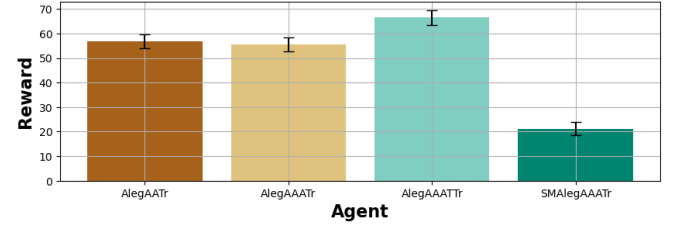
(d) Coordination



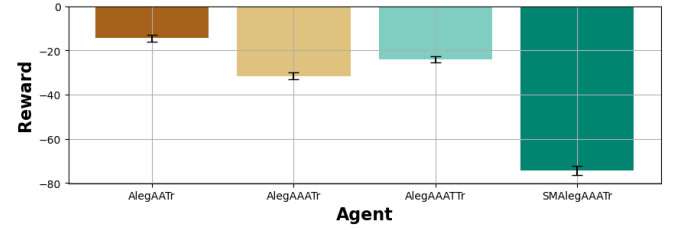
(e) Grid Stag Hunt



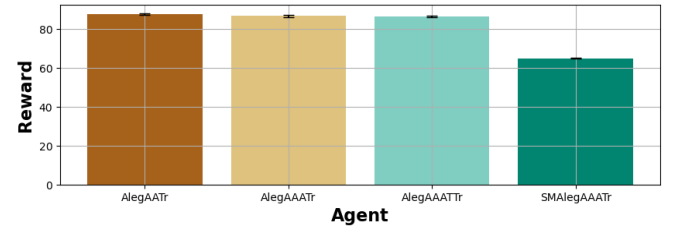
(a) Forex



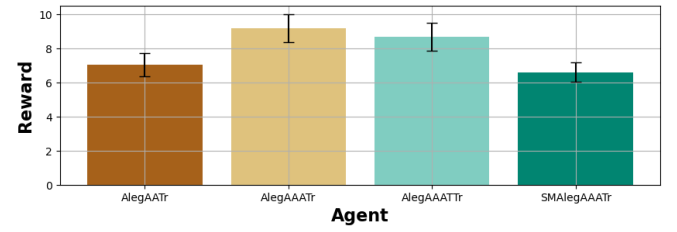
(b) Prisoner's



(c) Chicken



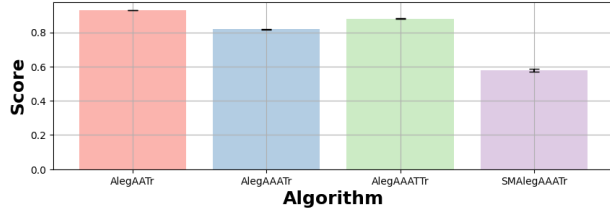
(d) Coordination



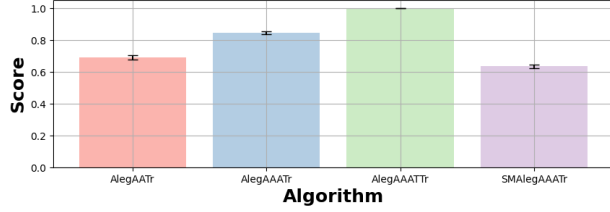
(e) Grid Stag Hunt

Fig. 5: Average results for each algorithm across the evaluation domains when AATention was trained only on chicken game data. Bars indicate standard errors.

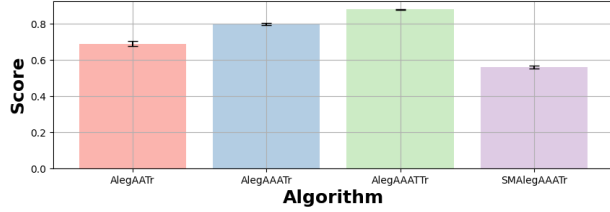
Fig. 6: Average results for each algorithm across the evaluation domains when AATention was trained on both JHG and chicken game data. Bars indicate standard errors.



(a) Average defect scores, with standard errors.

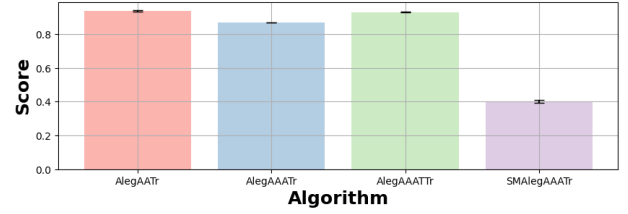


(b) Average cooperate scores, with standard errors.

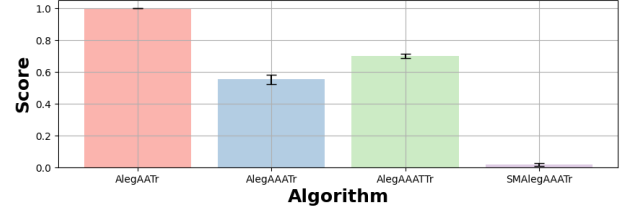


(c) Average adaptability scores, with standard errors.

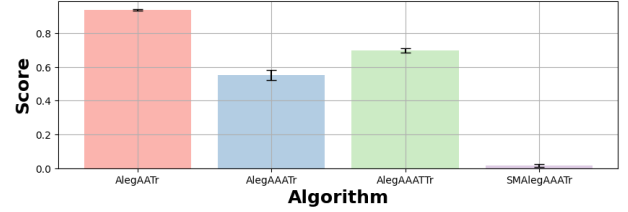
Fig. 7: Average defect, cooperative, and adaptability scores, respectively, with standard errors (black bars), for each algorithm in the repeated prisoner's dilemma when AATention was trained on data from both the JHG and repeated chicken game.



(a) Average defect scores, with standard errors.

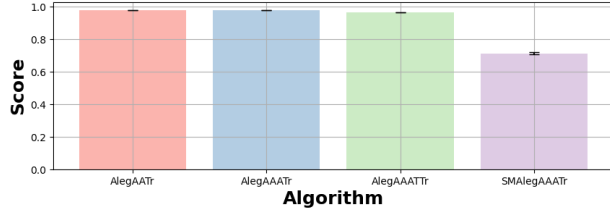


(b) Average cooperate scores, with standard errors.

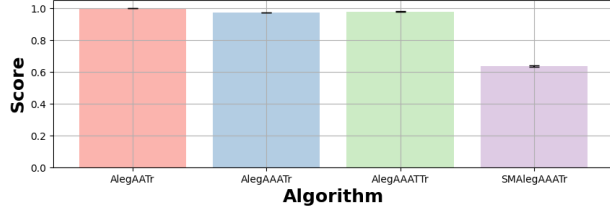


(c) Average adaptability scores, with standard errors.

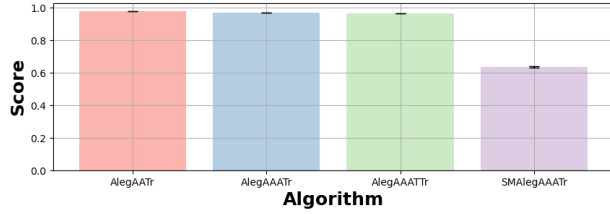
Fig. 8: Average defect, cooperative, and adaptability scores, respectively, with standard errors (black bars), for each algorithm in the repeated chicken game when AATention was trained on data from both the JHG and repeated chicken game.



(a) Average defect scores, with standard errors.

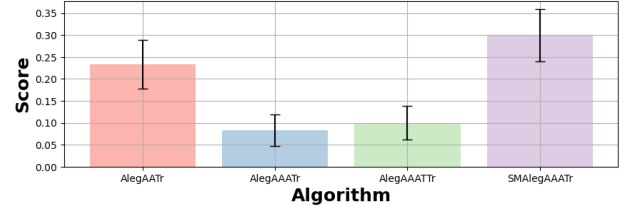


(b) Average cooperative scores, with standard errors.

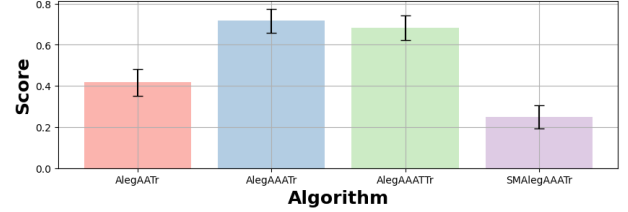


(c) Average adaptability scores, with standard errors.

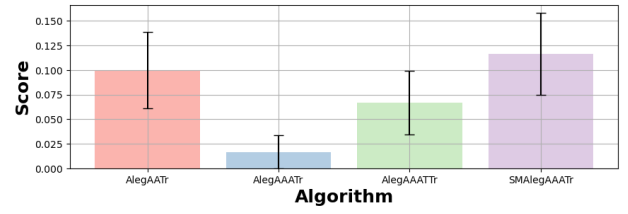
Fig. 9: Average defect, cooperative, and adaptability scores, respectively, with standard errors (black bars), for each algorithm in the repeated coordination game when AATention was trained on data from both the JHG and repeated chicken game.



(a) Average defect scores, with standard errors.



(b) Average cooperative scores, with standard errors.



(c) Average adaptability scores, with standard errors.

Fig. 10: Average defect, cooperative, and adaptability scores, respectively, with standard errors (black bars), for each algorithm in the Grid Stag Hunt domain when AATention was trained on data from both the JHG and repeated chicken game.